

SISTEMA DE GESTÃO DE ESTOQUE PARA MICROEMPRESAS DE VAREJO

FATEC-SP - ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Projeto Integrador I

Professor Orientador: Ana Travassos Ichihara

Integrantes: Caio Yao Zu Zhu, Cezar Hideak Ishie, Pedro Augusto Silva Santos,
João Pedro da Silva Martins, Artur Albertini Leal, Leonardo Gagliardi de Paula Rolandi.

São Paulo

2026

SUMÁRIO

1. Introdução	3
2. Fundamentação Teórica	4
3. Metodologia	5
4. Desenvolvimento	9
5. Código.....	16
6. Feedback.....	22
7. Apêndice A - Código fonte.....	23

1. INTRODUÇÃO

Este trabalho apresenta o desenvolvimento de um sistema de gestão de estoque voltado para microempresas do setor varejista. A crescente competitividade do mercado exige que pequenos negócios adotem soluções tecnológicas para melhorar o controle de seus processos internos.

O objetivo geral é desenvolver um sistema simples, acessível e eficiente para controle de entrada e saída de produtos. Entre os objetivos específicos, destacam-se: organizar o cadastro de produtos, automatizar o controle de estoque e gerar relatórios básicos para tomada de decisão.

A **comunidade** estudada é composta por micro empresas de varejo que, em sua maioria, ainda utilizam métodos manuais ou planilhas para controle de estoque, o que pode gerar erros e prejuízos.

A escolha do problema se justifica pela necessidade de digitalização desses negócios, contribuindo para maior organização, redução de perdas e aumento da eficiência operacional.

O trabalho está estruturado em seções que abordam a fundamentação teórica, metodologia e desenvolvimento do sistema proposto.

2. FUNDAMENTAÇÃO TEÓRICA E CIENTÍFICA

O desenvolvimento do sistema de controle de estoque proposto neste projeto está fundamentado em conceitos interligados de diferentes áreas da computação, que, em conjunto, possibilitam a construção de uma solução eficiente e adequada às necessidades das microempresas de varejo.

A **Engenharia de Software** atua como base estruturadora do projeto, orientando todas as etapas do desenvolvimento, desde o levantamento de requisitos até a implementação e testes do sistema. Por meio dessa abordagem, é possível garantir organização, qualidade e alinhamento da solução com os problemas identificados na pesquisa realizada.

Complementando esse processo, os conceitos de **Algoritmos e Lógica de Programação** são responsáveis pela definição das regras de funcionamento do sistema, como o controle de entrada e saída de produtos, atualização automática de quantidades e geração de relatórios. Esses elementos são essenciais para garantir a consistência e a confiabilidade das informações armazenadas. A linguagem **Python**, que está sendo ensinada nesse semestre, será usada para desenvolver este trabalho, possibilitando a funcionalidade do programa

A **Arquitetura de Computadores** contribui ao fornecer a compreensão de como os dados são processados e armazenados fisicamente, permitindo que o sistema seja pensado de forma eficiente em relação ao uso de memória e desempenho.

Em conjunto, os **Sistemas Operacionais** desempenham um papel fundamental ao gerenciar os recursos do sistema, garantindo que a aplicação funcione de maneira estável e segura durante sua execução. Será utilizado o **Windows 10** para desenvolver o projeto e o sistema, uma vez que é o SO mais comum e utilizado pelo público-alvo deste trabalho

Por fim, os **Sistemas de Informação** integram todos esses conceitos ao focar na organização, processamento e disponibilização dos dados de forma estratégica. No contexto deste projeto, isso se traduz em um sistema capaz de transformar dados de estoque em informações úteis, auxiliando os empreendedores na tomada de decisão, redução de erros e melhoria da gestão do negócio.

3. METODOLOGIA

A metodologia adotada neste trabalho tem como objetivo compreender e propor uma solução para os problemas relacionados ao controle de estoque em microempresas do setor varejista. Para isso, foi realizada uma **pesquisa aplicada**, com foco na análise de uma situação real.

Como parte da coleta de dados, foi implementado um **formulário direcionado à comunidade**, contendo perguntas relacionadas à gestão de estoque. Essa pesquisa teve como finalidade identificar as principais dificuldades enfrentadas pelos empreendedores, permitindo uma compreensão mais precisa do problema e orientando o desenvolvimento da solução proposta

3.1 Coleta de dados

A fim de coletar dados para analisar os requisitos referente ao projeto, a equipe decidiu redigir um formulário que consiste nas seguintes perguntas:

- Você possui dificuldades em organizar as suas mercadorias?*
- Como são registradas as entradas e saídas do seu estoque?*
- Você possui um sistema de integração de estoque?*
- Quais são seus canais de venda?*
- Quais informações você precisa armazenar de cada produto?*
- Atualmente qual é o maior problema que você possui com seu estoque?*
- Acredita que a implementação de um sistema de estoque ajudaria nessa tarefa?*

https://docs.google.com/forms/d/e/1FAIpQLSfhaWZ_i0stlWQriUBAfDsy-liEj4d4S393lnr4qfAlRpA7PQ/viewform

3.2 Informações a serem levantados:

3.2.1 Requisitos funcionais(RF):

Os requisitos funcionais descrevem as ações e comportamentos essenciais que o sistema deve executar para atender às necessidades da operação de varejo:

1. RF01 - Cadastro de Usuário: O sistema deve permitir o cadastro de usuário do sistema
2. RF02 - Cadastro de Produtos: O sistema deve permitir o registro de novos produtos contendo um identificador único (ID), descrição do nome, quantidade inicial em estoque e preço unitário.
3. RF03 - Atualização de Entrada: O sistema deve permitir a entrada de novas mercadorias, somando a quantidade inserida ao saldo do produto já existente no banco de dados.
4. RF04 - Registro de Saída/Venda: O sistema deve processar a venda de itens, calculando o valor total e abatendo a quantidade correspondente do saldo de estoque imediatamente.
5. RF05 - Consulta Rápida: O sistema deve fornecer uma funcionalidade de busca, permitindo localizar produtos pelo ID para verificar detalhes e o saldo disponível em tempo real.
6. RF06 - Geração de Relatórios: O sistema deve emitir um relatório consolidado contendo a listagem atualizada de todos os itens cadastrados, suas respectivas quantidades e preços, visando auxiliar o gestor no controle físico.

3.2.2 Requisitos Não Funcionais (RNF)

Os requisitos não funcionais determinam as restrições e as condições técnicas sob as quais o sistema operará de forma eficiente no ambiente da microempresa:

1. RNF01 - Compatibilidade (Sistema Operacional): A aplicação deve ser compatível e executável de forma nativa em computadores com sistema operacional Windows 10 ou superior.
2. RNF02 - Desempenho (Hardware): O software deve operar de forma fluida em computadores com recursos básicos (processadores de entrada como Intel Core i3 ou similares, a partir de 4 GB de memória RAM), visto que as máquinas de frente de caixa geralmente possuem baixo poder de processamento.
3. RNF03 - Usabilidade (Interface): A interface gráfica deve ser simples, intuitiva e direta, garantindo que usuários com pouca familiaridade tecnológica consigam realizar registros de vendas e entradas com um número mínimo de interações.

4. RNF04 - Tempo de Resposta: As atualizações de saldo e a busca por produtos devem ocorrer em tempo real (resposta inferior a 2 segundos), evitando gargalos e filas no momento do atendimento físico ao cliente.

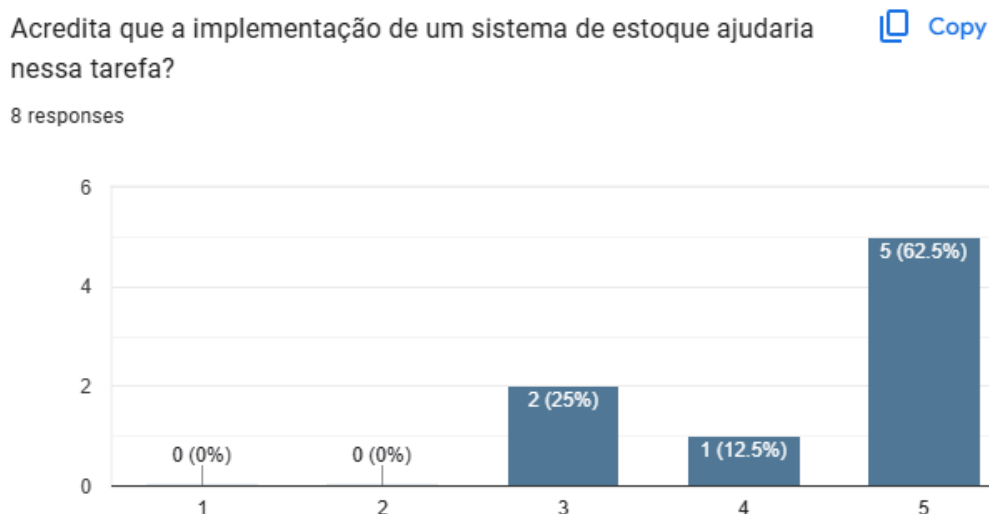
3.3.3 Regras de Negócio (RN)

As regras de negócio são as diretrizes restritivas que o sistema deve impor para garantir a integridade da operação comercial:

1. RN01 - Restrição de Estoque Negativo: O sistema deve bloquear sumariamente a conclusão de uma operação de venda caso a quantidade solicitada pelo usuário seja superior à quantidade disponível registrada no inventário atual.
2. RN02 - Exclusividade de Identificação: É estritamente necessário que todo produto cadastrado receba um ID único, não sendo permitida a duplicidade de códigos no banco de dados, para evitar conflitos na contagem multicanal.
3. RN03 - Centralização de Saldo: Todas as operações (entradas e vendas) devem refletir no mesmo banco central. O saldo exibido pelo sistema deve ser considerado a verdade absoluta e imediata para toda a operação da loja.

3.3 Análise de dados

Essa etapa é fundamental para transformar os dados brutos em informações relevantes, permitindo uma compreensão mais clara do problema e contribuindo diretamente para a definição de uma solução adequada e eficiente.



A análise do gráfico acima demonstra uma forte tendência positiva em relação à implementação de um sistema de controle de estoque. A maioria dos respondentes (62,5%) atribuiu nota máxima (5), indicando plena concordância de que um sistema auxiliará na gestão. Além disso, 25% atribuíram nota 3 e 12,5% nota 4, não havendo respostas negativas (notas 1 ou 2).

Esses resultados evidenciam que há uma **alta percepção de necessidade e aceitação** por parte dos entrevistados quanto à adoção de uma solução tecnológica para o controle de estoque. Mesmo entre os que não atribuíram nota máxima, ainda existe uma inclinação favorável, reforçando que a implementação de um sistema pode contribuir significativamente para a melhoria dos processos.

4. DESENVOLVIMENTO

Estão documentadas nesta seção, as etapas do processo de desenvolvimento do sistema de gestão de estoque integrado.

4.1 Levantamento do Problema

Segundo os dados fornecidos pela comunidade chegou-se na seguinte definição de um problema central:

"A ausência de uma gestão de inventário integrada e automatizada"

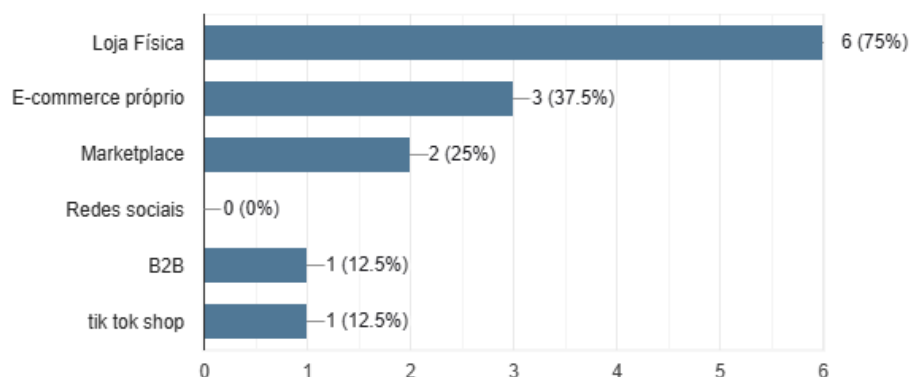
Hoje, muitos lojistas ainda enfrentam dificuldades porque dependem de controles manuais, como planilhas ou até processos sem registro algum. Mesmo quando têm sistemas, eles nem sempre são usados da forma ideal (devido a falta de praticidade), ou cobrem só uma parte da operação, como os marketplaces, deixando outras áreas de fora.

Com a presença em vários canais — loja física, online, B2B — essa falta de integração acaba virando o principal problema. As informações ficam espalhadas, não conversam entre si e dificultam ter uma visão clara do negócio. No dia a dia, isso se traduz em erros, desorganização e falta de controle sobre estoque e espaço.

Quais são seus canais de venda?

 Copy

8 responses



No fundo, tudo isso mostra a mesma coisa: sem uma solução integrada e automatizada, que centralize os dados em tempo real, a operação perde eficiência e as decisões ficam muito mais difíceis.

Justificativa :

Esse projeto faz sentido porque os próprios lojistas já enxergam o valor de uma solução melhor. Porém fica claro que não adianta criar “mais um sistema”. O que falta hoje é algo que conecte tudo: a loja física, o estoque e as vendas online, sem deixar as informações espalhadas. A ideia é justamente simplificar e trazer tudo para um lugar só, de forma organizada.

4.2 Definição da Solução/Ideia do Sistema

O projeto propõe o desenvolvimento de uma plataforma unificada e automatizada de gestão de estoque multicanal, projetada para centralizar as operações de varejo de microempresas, integrando loja física, e-commerce e vendas B2B em um único ambiente.

No uso real e diário, o sistema substituirá as planilhas manuais e os controles fragmentados por meio de uma interface simplificada e focada nas rotinas essenciais da equipe. A cada movimentação ou venda realizada em qualquer frente de caixa ou plataforma online, o sistema registrará a saída e sincronizará o inventário instantaneamente para toda a rede, eliminando o atrito operacional e o retrabalho.

Em seu cenário ideal, a solução não se posiciona apenas como mais um software de gestão, mas atua como a fonte única de verdade do negócio e um integrador invisível da operação. Ao consolidar informações que antes ficavam dispersas, o sistema garantirá ao gestor uma visão clara e em tempo real da empresa, prevenindo furos de estoque, otimizando o espaço físico e viabilizando tomadas de decisão mais ágeis e seguras.

4.3 Protótipo

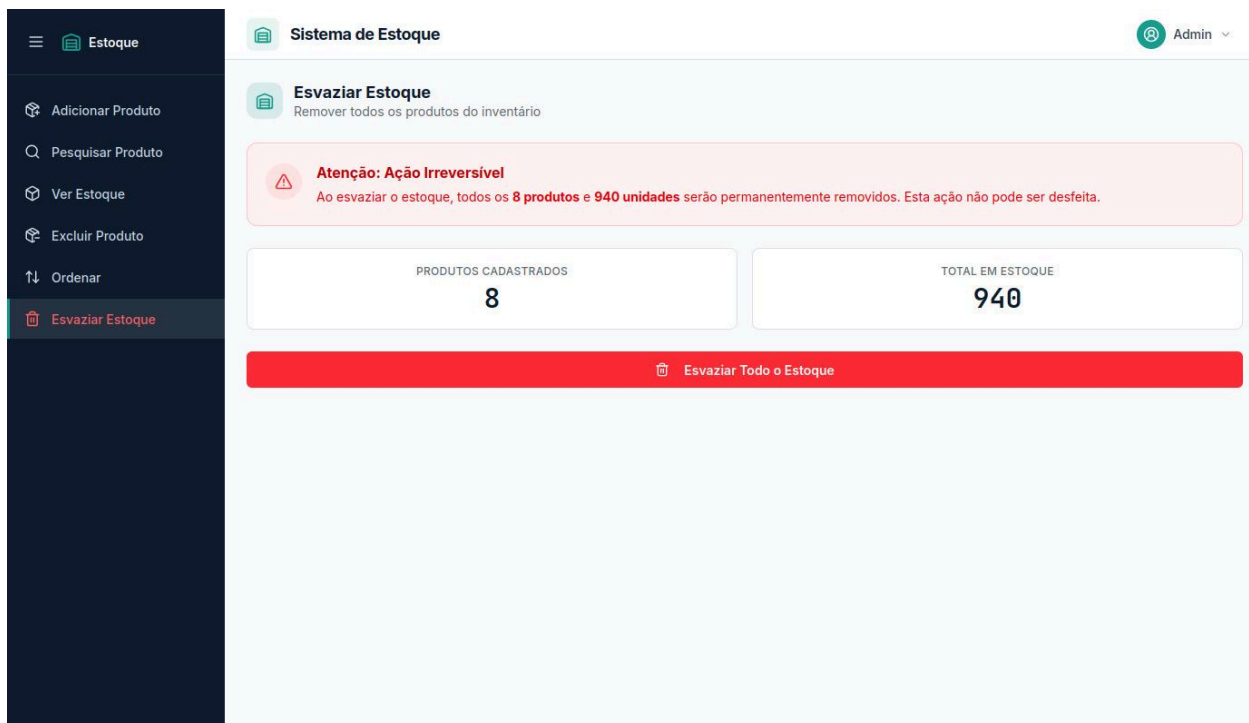
4.3.1 Análise do Protótipo do Sistema

O protótipo desenvolvido apresenta uma interface simples, organizada e voltada para facilitar o gerenciamento de estoque em microempresas do setor varejista. A estrutura visual foi planejada com foco na praticidade e na facilidade de utilização, permitindo que o usuário visualize e manipule informações de forma rápida e intuitiva.

ID	Produto	Preço	Quantidade	Ações
1	Papel A4	R\$ 0,25	250	
2	Caneta Esferográfica Azul	R\$ 1,50	120	
3	Caderno Universitário	R\$ 18,00	80	
4	Pasta Suspensa	R\$ 2,50	60	
5	Grampeador Médio	R\$ 35,00	35	
6	Clips 3/0	R\$ 0,04	200	
7	Fita Adesiva Transparente	R\$ 8,00	45	
8	Post-it 76x76mm	R\$ 9,00	150	

O sistema possui um menu lateral responsável por centralizar as principais funcionalidades relacionadas ao controle de estoque, contribuindo para uma navegação mais objetiva e organizada. Além disso, a disposição dos elementos na interface permite que diferentes operações sejam realizadas de maneira eficiente, sem a necessidade de múltiplas telas complexas.

Outro ponto relevante identificado no protótipo é a preocupação com a visualização das informações mais importantes do estoque, como quantidade de produtos e situação dos itens cadastrados. Isso auxilia diretamente no acompanhamento das mercadorias e na tomada de decisão dos usuários.



Z

Estoque

Adicionar Produto

Pesquisar Produto

Ver Estoque

Excluir Produto

Ordenar

Esvaziar Estoque

Sistema de Estoque

Admin

Pesquisar Produto

Busque por nome ou ID

Pesquisar por nome ou ID...

ID	Produto	Preço	Quantidade
1	Papel A4	R\$ 0,25	250
2	Caneta Esferográfica Azul	R\$ 1,50	120
3	Caderno Universitário	R\$ 18,00	80
4	Pasta Suspensa	R\$ 2,50	60
5	Grampeador Médio	R\$ 35,00	35
6	Clips 3/0	R\$ 0,04	200
7	Fita Adesiva Transparente	R\$ 8,00	45
8	Post-it 76x76mm	R\$ 9,00	150

Estoque

Adicionar Produto

Pesquisar Produto

Ver Estoque

Excluir Produto

Ordenar

Esvaziar Estoque

Sistema de Estoque

Admin

Estoque Atual

Lista de produtos cadastrados

Atualizar

ID	Produto	Preço	Quantidade
1	Papel A4	R\$ 0,25	250
2	Caneta Esferográfica Azul	R\$ 1,50	120
3	Caderno Universitário	R\$ 18,00	80
4	Pasta Suspensa	R\$ 2,50	60
5	Grampeador Médio	R\$ 35,00	35
6	Clips 3/0	R\$ 0,04	200
7	Fita Adesiva Transparente	R\$ 8,00	45
8	Post-it 76x76mm	R\$ 9,00	150

TOTAL DE PRODUTOS

8

TOTAL EM ESTOQUE

940

Adicionar Novo Produto

Nome do Produto

Digite o nome do produto

Preço (R\$)

Digite o preço

Quantidade

Digite a quantidade

Adicionar Produto

Limpar Campos

Estoque

Adicionar Produto

Pesquisar Produto

Ver Estoque

Excluir Produto

Ordenar

Esvaziar Estoque

Sistema de Estoque

Admin

Ordenar Estoque

Organize os produtos por campo

Ordenar por: ID Nome Quantidade

Crescente Decrescente

ID	Produto	Preço	Quantidade
1	Papel A4	R\$ 0,25	250
2	Caneta Esferográfica Azul	R\$ 1,50	120
3	Caderno Universitário	R\$ 18,00	80
4	Pasta Suspensa	R\$ 2,50	60
5	Grampeador Médio	R\$ 35,00	35
6	Clips 3/0	R\$ 0,04	200
7	Fita Adesiva Transparente	R\$ 8,00	45
8	Post-it 76x76mm	R\$ 9,00	150

Estoque

Adicionar Produto

Pesquisar Produto

Ver Estoque

Excluir Produto

Ordenar

Esvaziar Estoque

Sistema de Estoque

Admin

Ver Estoque

Visualização completa do inventário

Atualizar

ID	Produto	Preço	Quantidade
1	Papel A4	R\$ 0,25	250
2	Caneta Esferográfica Azul	R\$ 1,50	120
3	Caderno Universitário	R\$ 18,00	80
4	Pasta Suspensa	R\$ 2,50	60
5	Grampeador Médio	R\$ 35,00	35
6	Clips 3/0	R\$ 0,04	200
7	Fita Adesiva Transparente	R\$ 8,00	45
8	Post-it 76x76mm	R\$ 9,00	150

TOTAL DE PRODUTOS

8

TOTAL EM ESTOQUE

940

4.3.2 Considerações sobre o Protótipo

O protótipo desenvolvido demonstra uma proposta funcional alinhada às necessidades identificadas durante a pesquisa realizada com os empreendedores. A interface prioriza simplicidade, organização e facilidade de utilização, características importantes para microempresas que necessitam de soluções práticas para o gerenciamento de estoque.

Além disso, as funcionalidades identificadas contribuem para melhorar o controle de mercadorias, reduzir erros operacionais e otimizar os processos relacionados ao armazenamento e movimentação de produtos.

5. CÓDIGO

5.1 ESTRUTURA INICIAL DO SISTEMA

Este segmento apresenta a instanciação das bibliotecas fundamentais utilizadas pelo sistema, bem como o mecanismo responsável pela verificação e garantia da persistência dos dados em arquivo de formato CSV (Comma-Separated Values). A rotina assegura que a infraestrutura básica de armazenamento esteja disponível antes da execução das demais operações.

```
import pandas as pd
import os

# Verifica a existência do arquivo de dados
if not os.path.exists("estoque.csv"):
    # Inicializa a estrutura com os atributos fundamentais
    df = pd.DataFrame(columns=["id", "produto", "quantidade",
                              "preco_unitario"])

    # Cria o arquivo CSV para persistência local
    df.to_csv("estoque.csv", index=False)

# Carrega os dados para processamento em memória
df_resultado = pd.read_csv("estoque.csv")
```

5.2 ESTRUTURA DE CONTROLE E FLUXO OPERACIONAL

Esta seção define o controle da execução contínua do sistema. Através de um laço de repetição (while), a aplicação mantém a sessão ativa e exibe o roteamento das funcionalidades disponíveis ao usuário. O tratamento de exceções (try-except) é implementado para garantir a estabilidade do programa frente a entradas inválidas.


```

res = ""

while res != "n":
    print("
        SISTEMA DE ESTOQUE
    ")

    try:
        # Interface de navegação do usuário
        escolha = int(input(
            "    [1] Adicionar produto
"
            "    [2] Pesquisar produto
"
            "    [3] Mostrar o estoque inteiro
"
            "    [4] Excluir produto
"
            "    [5] Ordenar estoque
"
            "    [6] Esvaziar estoque
"
            "    [7] Atualizar preço
"
            "    [8] Sair

Escolha uma opção: "
        )) except
        ValueError:
            print("
Erro: Digite uma opção numérica válida.
")

            continue

```

5.3 IMPLEMENTAÇÃO DO CADASTRO DE PRODUTOS

A rotina a seguir é responsável por gerenciar a identificação e o cadastro de produtos. A implementação utiliza uma lógica de auto-incremento para a geração de identificadores únicos (IDs) e aplica métodos de sanitização de strings para padronizar a entrada textual, mitigando erros decorrentes de digitação e facilitando buscas futuras.

```
# Geração do ID incremental automático
if not df_resultado.empty:
    id_prod = int(df_resultado["id"].max()) + 1
else:
    id_prod = 1

produto = "" while
not produto:
    # Sanitização e padronização da entrada de texto
    produto = input("
    Digite o nome do produto: ").strip().replace(" ", "_").lower()

    if not produto:
        print("O nome do produto não pode estar vazio!")

# Verificação de duplicidade no estoque produto_ja_existe =
produto in df_resultado["produto"].values
```

5.4 VALIDAÇÃO DE ENTRADAS

O sistema implementa mecanismos robustos de validação tipográfica e lógica para garantir a integridade e a consistência dos dados numéricos fornecidos pelo usuário, impedindo o processamento de quantitativos negativos ou nulos.

```

qnt = 0 while
qnt <= 0:
    try:
        # Validação da quantidade fornecida pelo usuário
        qnt = int(input("
Digite a quantidade: "))
    if qnt <= 0:
        print("A quantidade deve ser maior do que 0.")
    except ValueError:
        print("Erro: Digite apenas números inteiros.")

```

5.5 ATUALIZAÇÃO DINÂMICA DO ESTOQUE

Ao identificar que um produto já compõe a base de dados do estoque, a aplicação executa uma atualização quantitativa do tipo in-place. O algoritmo localiza a linha correspondente através de uma máscara condicional no DataFrame e incrementa a quantidade disponível, sincronizando imediatamente o estado com o arquivo físico.

```

# Incrementa a quantidade utilizando filtragem condicional
df_resultado.loc[df_resultado["produto"] == produto, "quantidade"]
+= qnt

# Salva as alterações no arquivo físico
df_resultado.to_csv("estoque.csv", index=False)

```

5.6 INSERÇÃO DE NOVOS REGISTROS

Para a entrada de novos itens que ainda não constam na base, o sistema estrutura um dicionário de dados convertido em DataFrame. Em seguida, realiza a operação de inserção (append) diretamente no arquivo CSV, procedendo com a atualização do repositório em memória principal para garantir o sincronismo estrutural.

```

# Criação de um novo DataFrame para o novo registro
nova_linha = pd.DataFrame([
    "id": id_prod,
    "produto": produto,
    "quantidade": qnt,
    "preco_unitario": preco
])

# Anexa o novo registro ao final do arquivo CSV
nova_linha.to_csv(
    "estoque.csv",
    mode="a",
    header=False,
    index=False
)

# Sincroniza a memória com o arquivo atualizado
df_resultado = pd.read_csv("estoque.csv")

```

5.7 CONSULTA DE PRODUTOS

A funcionalidade de pesquisa isola registros específicos através da aplicação de filtros lógicos diretos sobre o vetor da coluna principal. A extração avalia a existência da correspondência e retorna o subconjunto (subset) localizado para o usuário.

```

# Procura pelo produto especificado no DataFrame df_produto =
df_resultado[df_resultado["produto"] == produto]

# Exibe o resultado com base na existência do item
if df_produto.empty:
    print("
Produto não localizado no estoque.")
else:
    print(df_produto)

```

5.8 REMOÇÃO DE PRODUTOS

O método de exclusão implementado opera por meio de reconstrução de dados (remoção lógica adaptada). O DataFrame é reestruturado de modo a conter exclusivamente os itens divergentes daquele indicado para exclusão. O resultado substitui a matriz original, concretizando a exclusão sistêmica.

```
# Reconstrução do DataFrame excluindo o produto selecionado df_filtrado  
= df_resultado[df_resultado["produto"] != produto]  
  
# Atualiza o arquivo de persistência externa  
df_filtrado.to_csv("estoque.csv", index=False)
```

5.9 ORDENAÇÃO DO ESTOQUE

A arquitetura possibilita a reorganização vetorial das informações apresentadas. Utilizando o método interno de ordenação da biblioteca Pandas, os dados são reordenados com base no atributo "quantidade", adotando a disposição decrescente como padrão analítico.

```
# Ordenação decrescente baseada na quantidade em estoque df_resultado =  
df_resultado.sort_values(by="quantidade", ascending=False)  
  
# Atualiza o arquivo físico com a nova ordenação  
df_resultado.to_csv("estoque.csv", index=False)
```

5.10 ATUALIZAÇÃO DE PREÇOS

O módulo de precificação viabiliza a mutabilidade do atributo financeiro de um item cadastrado. Identificado o registro por meio do seu indexador textual, o campo referente ao preço unitário é sobrescrito pelo novo valor monetário submetido.

```
# Atualiza o preço unitário por filtragem condicional  
df_resultado.loc[df_resultado["produto"] == produto, "preco_unitario"]  
= novo_preco  
  
# Persiste a alteração no arquivo físico  
df_resultado.to_csv("estoque.csv", index=False)
```

5.11 ENCERRAMENTO DA APLICAÇÃO

A diretiva de encerramento garante a interrupção segura do laço de controle principal (break statement). Essa ação finaliza as operações em memória, assegurando que nenhum processamento adicional comprometa os dados previamente salvos e consolidando a desocupação dos recursos de máquina em uso.

```
elif escolha == 8:  
    print("  
Sessão encerrada. Obrigado por utilizar o sistema!") break
```

6. FEEDBACK

As telas do protótipo apresentam uma interface moderna, organizada e de fácil utilização. O menu lateral facilita a navegação, enquanto as tabelas e informações estão bem distribuídas e claras.

O uso das cores ajuda na identificação rápida dos produtos e quantidades em estoque. Além disso, as telas mantêm um padrão visual consistente, transmitindo profissionalismo e melhorando a experiência do usuário.

De modo geral, o protótipo demonstra boa usabilidade, organização e praticidade.

APÊNDICE A – CÓDIGO FONTE COMPLETO

O apêndice a seguir apresenta o código-fonte completo utilizado no desenvolvimento do sistema de estoque.

```
import pandas as pd
import os

# Verificação de persistência e inicialização do schema do banco de dados (CSV)
if not os.path.exists("estoque.csv"):
    df = pd.DataFrame(
        columns=["id", "produto", "quantidade", "preco_unitario"])
    df.to_csv("estoque.csv", index=False)

# Carregamento do estado inicial do estoque para a memória
df_resultado = pd.read_csv("estoque.csv")

res = ""
while res != "n":
    print("\n", "="*10, "Bem vindo ao sistema de estoque para microempresas", "="*10,
          "\n")

    try:
        # Interface de roteamento principal
        escolha = int(input(
            "\n [1] Adicionar produto \n [2] Pesquisar produto \n [3] Mostrar o estoque inteiro \n [4] Excluir produto \n [5] Ordenar estoque \n [6] Esvaziar estoque \n [7] Atualizar preço \n [8] Sair \n\n ... "))

    except ValueError:
        print("\nDigite uma opção válida\n")
```

continue

```
# =====  
# [1] ADICIONAR OU INCREMENTAR PRODUTO  
# =====  
if escolha == 1:  
    # Lógica de auto-incremento de ID baseada no valor máximo atual do índice  
    if not df_resultado.empty:  
        id_prod = int(df_resultado["id"].max()) + 1  
    else:  
        id_prod = 1  
  
    produto = ""  
    while not produto:  
        # Normalização da string de entrada para padronizar as chaves de busca  
        produto = input("\nDigite o nome do produto: ").strip().replace(  
            " ", "_").lower()  
        if not produto:  
            print("O nome do produto não pode ser vazio!")  
  
    # Validação de existência prévia do produto no array de valores da coluna  
    produto_ja_existe = produto in df_resultado["produto"].values  
  
    qnt = 0  
    while qnt <= 0:  
        try:  
            qnt = int(input("\nDigite a quantidade: "))  
            if qnt <= 0:
```



```

        print("A quantidade tem que ser maior que 0")
except ValueError:
    print("\nDigite apenas números\n")

if produto_ja_existe:
    # Atualização in-place da quantidade usando localização por máscara booleana
    df_resultado.loc[df_resultado["produto"]
                     == produto, "quantidade"] += qnt
    print(
        f"\nEstoque de '{produto}' atualizado com sucesso (Quantidade somada)!\n")

# Sincronização do estado atualizado com o arquivo físico
df_resultado.to_csv("estoque.csv", index=False)

print("\n", 34*"=")
print(df_resultado)
print(34*"=", "\n")
continue

preco = 0
while preco <= 0:
    try:
        preco = float(input("\nDigite o preço: \n"))
        if preco <= 0:
            print("O preço tem que ser maior que 0")
    except ValueError:
        print("\nDigite apenas números\n")

```

```

# Criação de um novo registro e inserção em modo append ('a') no CSV
nova_linha = pd.DataFrame([
    "id": id_prod,
    "produto": produto,
    "quantidade": qnt,
    "preco_unitario": preco})
nova_linha.to_csv("estoque.csv", mode="a", header=False, index=False)

# Recarregamento do DataFrame para garantir paridade com o arquivo físico
df_resultado = pd.read_csv("estoque.csv")

print("\n", 41*"=")
print(df_resultado)
print(41*"=", "\n")

# =====
# [2] PESQUISAR PRODUTO
# =====
elif escolha == 2:
    produto = ""
    while not produto:
        produto = input("\nDigite o nome do produto: \n").strip().replace(
            " ", "_").lower()

# Extração de um subset do DataFrame usando filtro condicional
df_produto = df_resultado[df_resultado["produto"] == produto]

if df_produto.empty:

```

```

        print("\nNão está no estoque")
    else:
        print("\n", 41* "=")
        print(df_produto)
        print(41* "=", "\n")

# =====
# [3] MOSTRAR ESTOQUE COMPLETO
# =====

elif escolha == 3:
    print("\n", 41* "=")
    if df_resultado.empty:
        print("O estoque está vazio")
    else:
        print(df_resultado)
        print(41* "=", "\n")

# =====
# [4] DELETAR PRODUTO (TOTAL OU PARCIAL)
# =====

elif escolha == 4:
    if df_resultado.empty:
        print("\n", 41* "=")
        print("\nO estoque está vazio\n")
        print(41* "=", "\n")
    else:
        print("\n", 41* "=")
        print(df_resultado)

```

```

print(41*"=", "\n")

produto = ""
while not produto:
    produto = input("Digite o nome do produto para deletar: ").strip().replace(
        " ", "_").lower()

if produto not in df_resultado["produto"].values:
    print("\nProduto não encontrado no estoque.")
    continue

try:
    ex = int(input(
        "\n[1] Deletar todo o estoque de um produto\n[2] Deletar fragmentos \n\n ...
"))
except ValueError:
    print("\nDigite uma opção válida\n")
    continue

if ex == 1:
    # Remoção lógica: sobrescreve o DataFrame com todos os registros, exceto
    o selecionado
    df_filtrado = df_resultado[df_resultado["produto"] != produto]

    if len(df_filtrado) == len(df_resultado):
        print("Produto não encontrado")
    else:
        df_filtrado.to_csv("estoque.csv", index=False)
        df_resultado = df_filtrado

```

```

print("\nProduto deletado com sucesso!")

elif ex == 2:
    # Extração do valor escalar atual da quantidade para validação
    qnt_atual = df_resultado.loc[df_resultado["produto"]
                                == produto, "quantidade"].values[0]

    while True:
        try:
            qnt_ex = int(input("\nDeseja excluir quantos?\n"))
            if qnt_ex <= 0:
                print("A quantidade a excluir deve ser maior que 0.")
                continue

            # Validação de lower bound para evitar quantidade negativa
            if qnt_ex > qnt_atual:
                print(
                    f"Você só tem {qnt_atual} unidades desse produto. Tente
novamente.")
                continue

            break
        except ValueError:
            print("Digite um número válido")

    # Subtração do valor na célula específica e expurgo de registros zerados
    df_resultado.loc[df_resultado["produto"]
                    == produto, "quantidade"] -= qnt_ex
    df_resultado = df_resultado[df_resultado["quantidade"] > 0]

```

```

df_resultado.to_csv("estoque.csv", index=False)

print("\n", 41*"=")
print(df_resultado)
print(41*"=", "\n")

# =====
# [5] ORDENAR ESTOQUE
# =====

elif escolha == 5:
    while True:
        try:
            ordenacao = int(input(
                "\n [1] Ordenar por ID \n [2] Ordenar por produto \n [3] Ordenar por
quantidade \n [4] Ordenar por preço \n\n ... "))
            break
        except ValueError:
            print("Digite opções válidas")

    # Reordenação do DataFrame utilizando a função sort_values, com
ascending=False para numéricos de negócio

    if ordenacao == 1:
        df_resultado = df_resultado.sort_values(by="id")
        print("\n", 41*"=")
        print(df_resultado)
        print(41*"=", "\n")
        df_resultado.to_csv("estoque.csv", index=False)

```

```

elif ordenacao == 2:
    df_resultado = df_resultado.sort_values(by="produto")
    print("\n", 41*"=")
    print(df_resultado)
    print(41*"=", "\n")
    df_resultado.to_csv("estoque.csv", index=False)

```

```

elif ordenacao == 3:
    df_resultado = df_resultado.sort_values(
        by="quantidade", ascending=False)
    print("\n", 41*"=")
    print(df_resultado)
    print(41*"=", "\n")
    df_resultado.to_csv("estoque.csv", index=False)

```

```

elif ordenacao == 4:
    df_resultado = df_resultado.sort_values(
        by="preco_unitario", ascending=False)
    print("\n", 41*"=")
    print(df_resultado)
    print(41*"=", "\n")
    df_resultado.to_csv("estoque.csv", index=False)

```

```

else:
    print("\nDigite números válidos\n")

```

```

# =====
# [6] Esvaziar Estoque
# =====

```

```

elif escolha == 6:
    # Reset total do schema e sobrescrita do CSV
    df_resultado = pd.DataFrame(
        columns=["id", "produto", "quantidade", "preco_unitario"])
    df_resultado.to_csv("estoque.csv", index=False)
    print("\n", 41*"=")
    print("\nO estoque está vazio\n")
    print(41*"=", "\n")

# =====
# [7] ATUALIZAR PREÇO
# =====

elif escolha == 7:
    if df_resultado.empty:
        print("\n", 41*"=")
        print("\nO estoque está vazio\n")
        print(41*"=", "\n")
    else:
        print("\n", 41*"=")
        print(df_resultado)
        print(41*"=", "\n")

        produto = ""
        while not produto:
            produto = input("Digite o nome do produto para atualizar o preço: ").strip(
                ).replace(" ", "_").lower()

        if produto not in df_resultado["produto"].values:

```



```

        print("\nProduto não encontrado")
        continue

    novo_preco = 0
    while novo_preco <= 0:
        try:
            novo_preco = float(input("\nDigite o novo preço: \n"))
            if novo_preco <= 0:
                print("O preço tem que ser maior que 0")
        except ValueError:
            print("\nDigite apenas números\n")

    # Mutação da célula 'preco_unitario' no registro correspondente à máscara do
    produto
    df_resultado.loc[df_resultado["produto"] ==
                     produto, "preco_unitario"] = novo_preco
    df_resultado.to_csv("estoque.csv", index=False)

    print(f"\nPreço do produto '{produto}' atualizado com sucesso!\n")

    print("\n", 41*"=")
    print(df_resultado)
    print(41*"=", "\n")

    # =====
    # [8] FINALIZAR EXECUÇÃO
    # =====

    elif escolha == 8:
        break

```

```
else:
    print("\nDigite uma opção válida\n")

# Avaliação do loop principal para manutenção da sessão
while True:
    res = input("\nDeseja fazer algo a mais? s|n \n").strip().lower()

    if res not in ["n", "s"]:
        print("\nResponda com s ou n \n")
    else:
        break

print("\n", 41*"=")
print("Obrigado por utilizar nosso sistema!")
print(41*"=", "\n")
```